1. Creating a GitHub Account and Repository

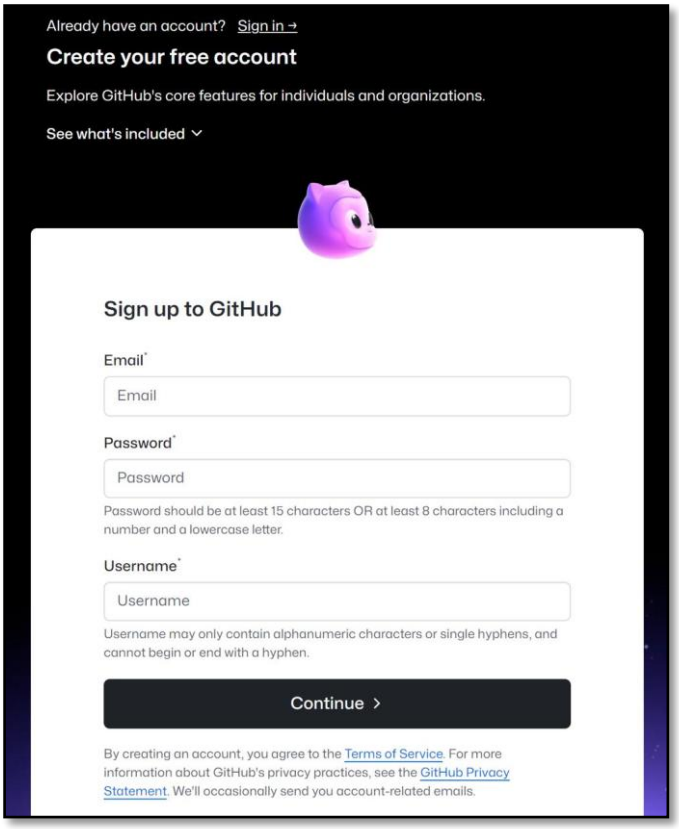
1. Objective

To create a GitHub account and a new repository for storing, managing, and version-controlling source code using GitHub.

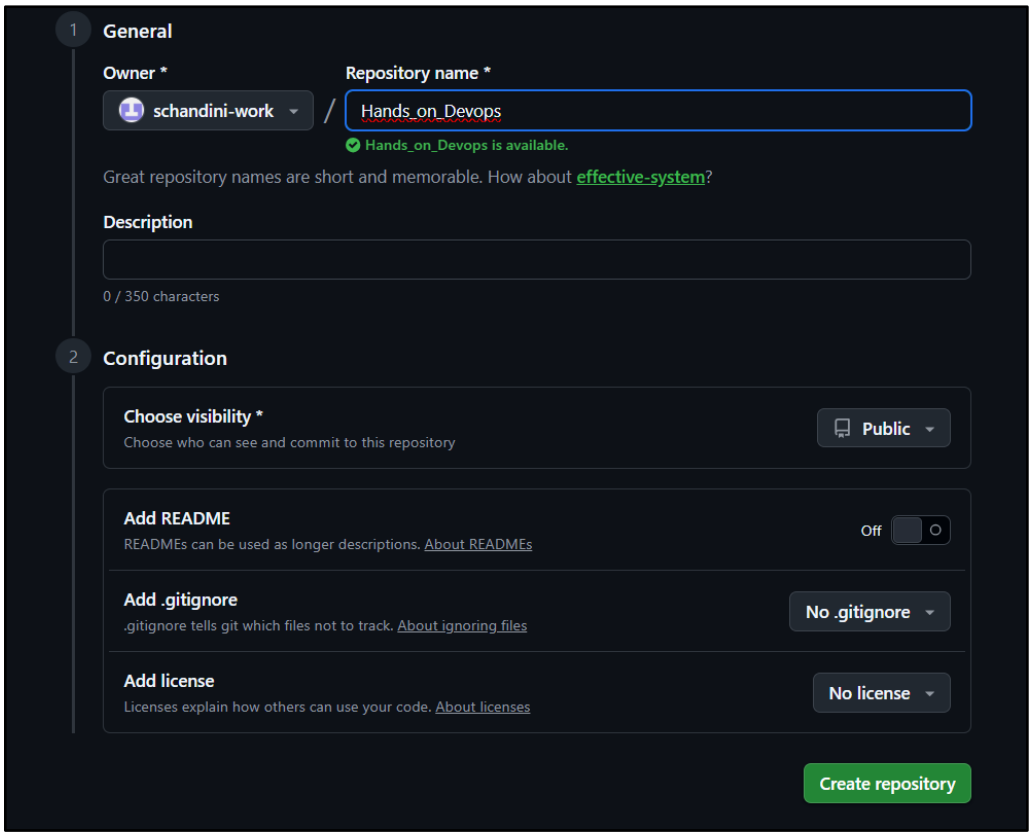
2. Step-by-Step Procedure

1. Open the official GitHub website: https://github.com/.
2. Click Sign up to create a new account.
3. Enter your email address, create a password, and choose a username.
4. Complete the required verification process and sign in to GitHub.
5. Click the + icon in the upper-right corner and select New repository.
6. Enter a repository name, such as Azure-Pipeline-Demo.
7. Add a short description, such as Sample repository for Azure DevOps pipeline.
8. Choose Public or Private visibility.
9. Optionally, select Add a README file.
10. Click Create repository.
11. Verify that the repository has been created successfully.

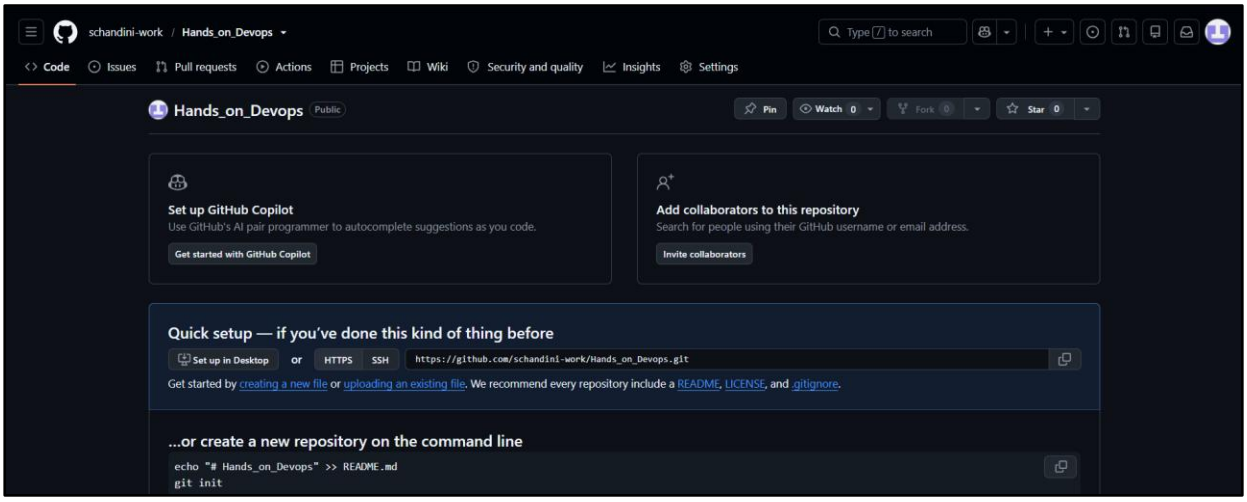
3. Screenshots



Screenshot 1.1: GitHub Sign-Up Page



Screenshot 1.2: Create Repository



Screenshot 1.3: Created Repository

4. Expected Result / Outcome

A GitHub account and a new repository are created successfully. The repository can be used to store source code, track changes, collaborate with other developers, and connect to Azure DevOps pipelines.

2. Creating an Azure DevOps Organization

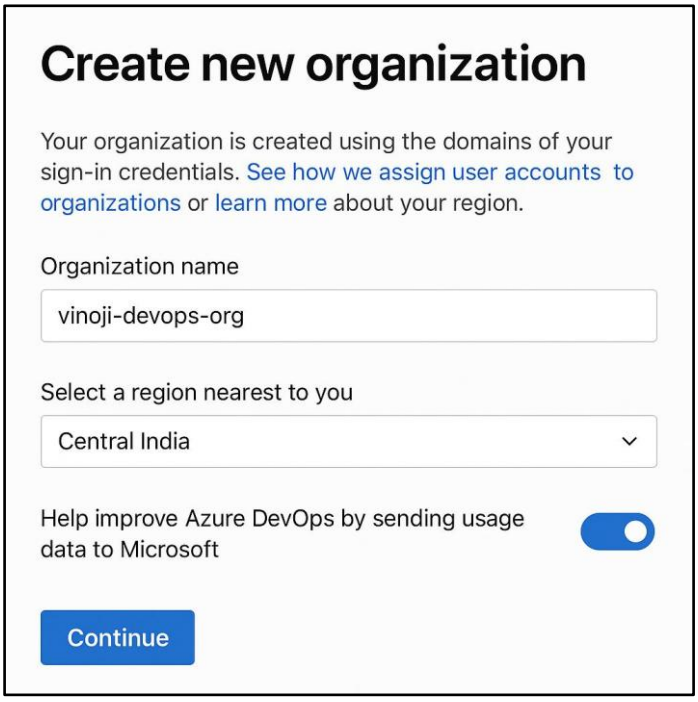
1. Objective

To create an Azure DevOps organization for managing software projects, source code, work items, builds, and deployment pipelines.

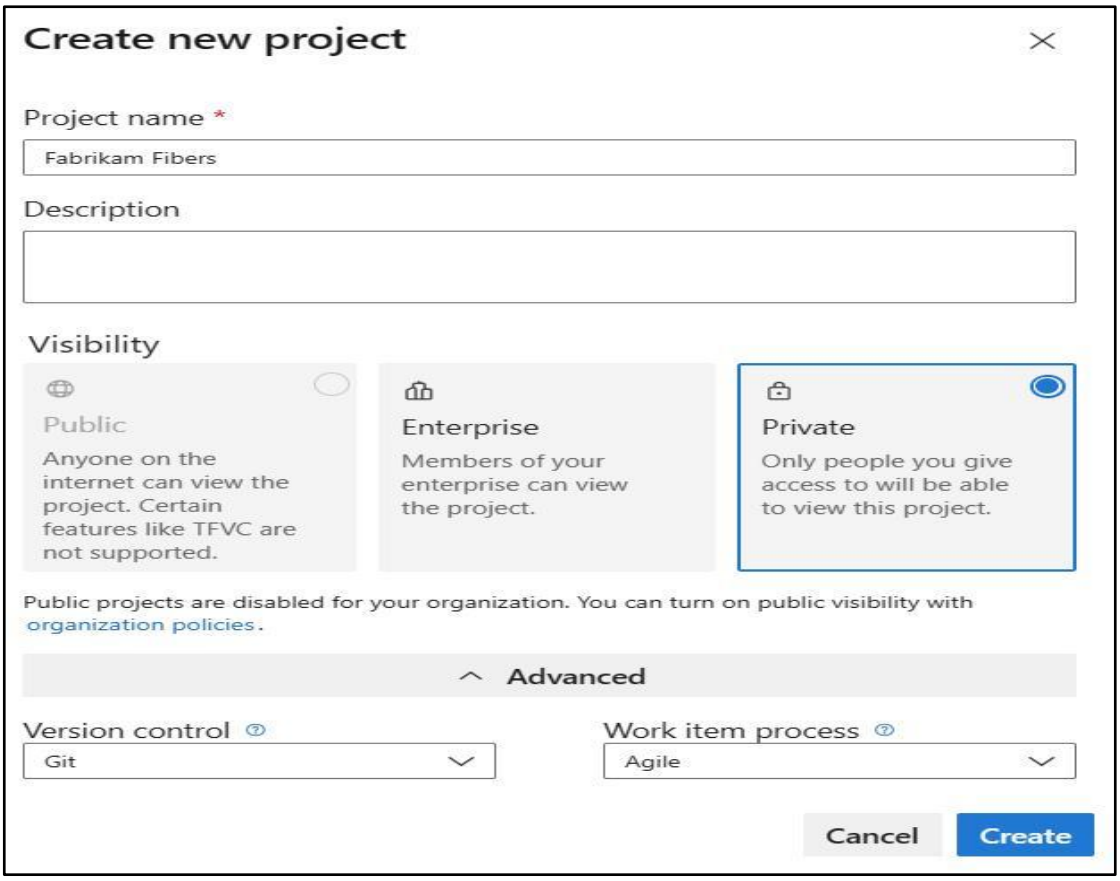
2. Step-by-Step Procedure

1. Open the official Azure DevOps website: https://dev.azure.com/.
2. Sign in using a Microsoft account.
3. If prompted, complete the required account setup.
4. Select the option to create a new organization.
5. Enter a unique organization name.
6. Choose the appropriate hosting region.
7. Complete any verification or setup steps displayed.
8. Open the newly created organization.
9. Click New project.
10. Enter a project name, such as AzurePipelineProject.
11. Choose the visibility setting and version control option.
12. Click Create.
13. Verify that the project dashboard opens successfully.

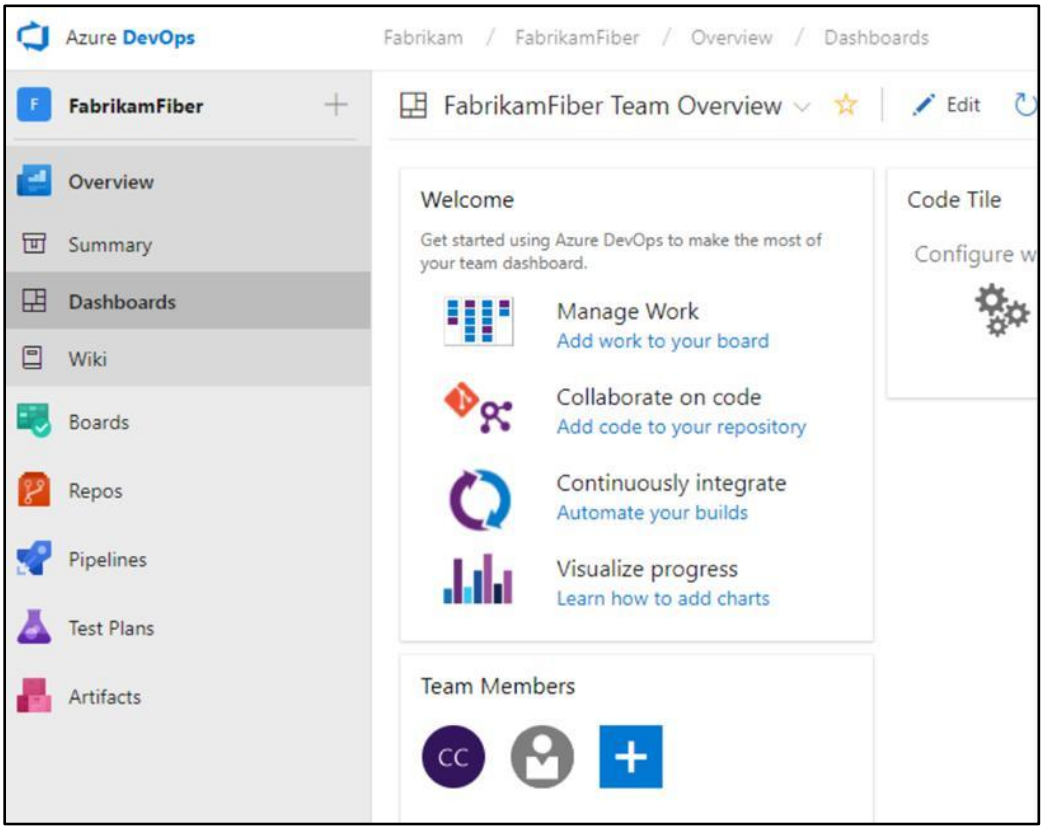
3. Screenshots



Screenshot 2.1: Create Organization



Screenshot 2.2: Create Project



Screenshot 2.3: Project Dashboard

4. Expected Result / Outcome

An Azure DevOps organization and project are created successfully. The project provides access to services such as Repos, Pipelines, Boards, and other development tools.

---

3. Creating a New Azure Pipeline

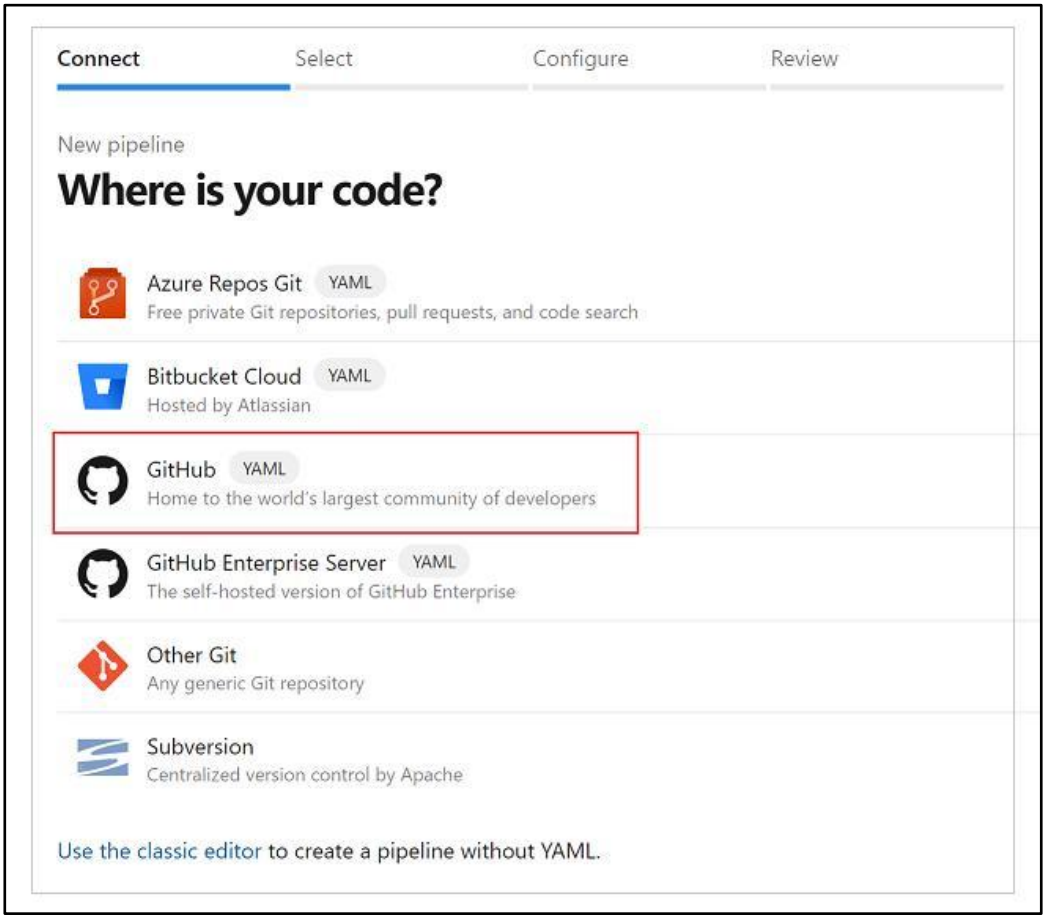
1. Objective

To create a new Azure Pipeline and connect it to a GitHub repository for automating the build process.

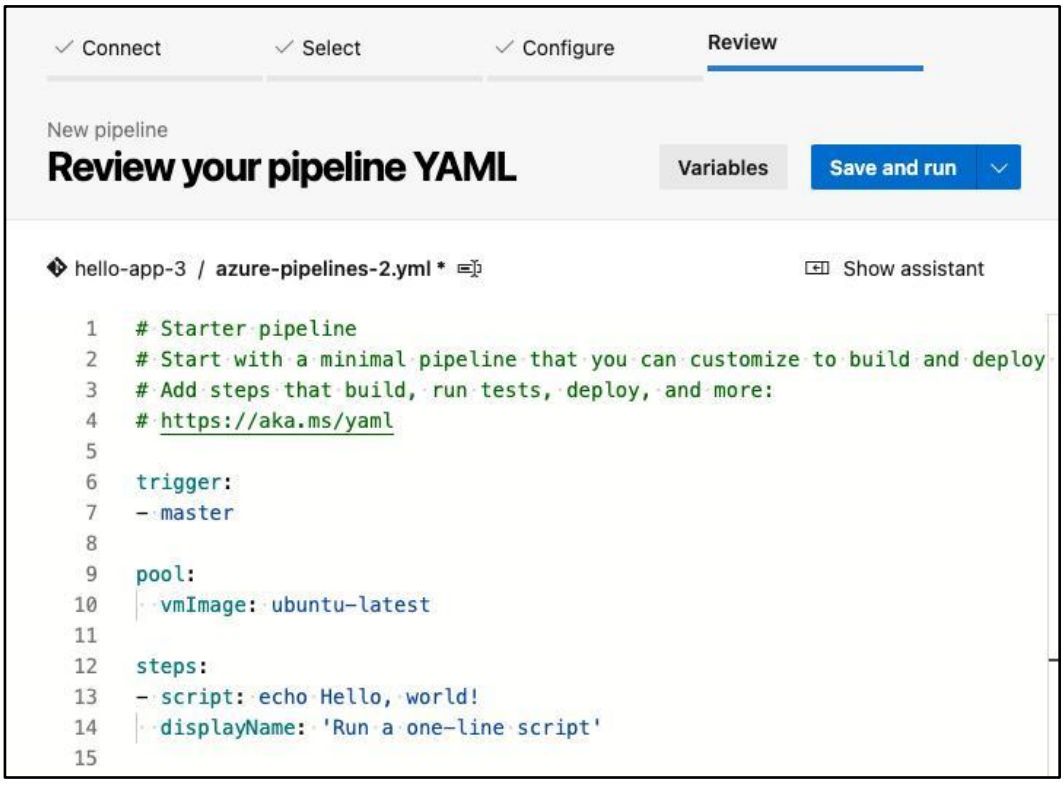
2. Step-by-Step Procedure

1. Sign in to Azure DevOps and open the project created in Experiment 2.
2. Select Pipelines from the left-side menu.
3. Click Create Pipeline or New pipeline.
4. Select GitHub as the source code provider.
5. Authorize Azure DevOps to access GitHub if prompted.
6. Select the repository created in Experiment 1, such as Azure-Pipeline-Demo.
7. Select Starter pipeline if Azure DevOps offers the configuration choices.
8. Azure DevOps opens the YAML editor with a default pipeline configuration.
9. Review the generated azure-pipelines.yml file.
10. Click Save and run.
11. Enter a commit message if requested and commit the YAML file to the repository.
12. Allow Azure DevOps to queue and execute the pipeline.
13. Open the pipeline run to view its status and logs.

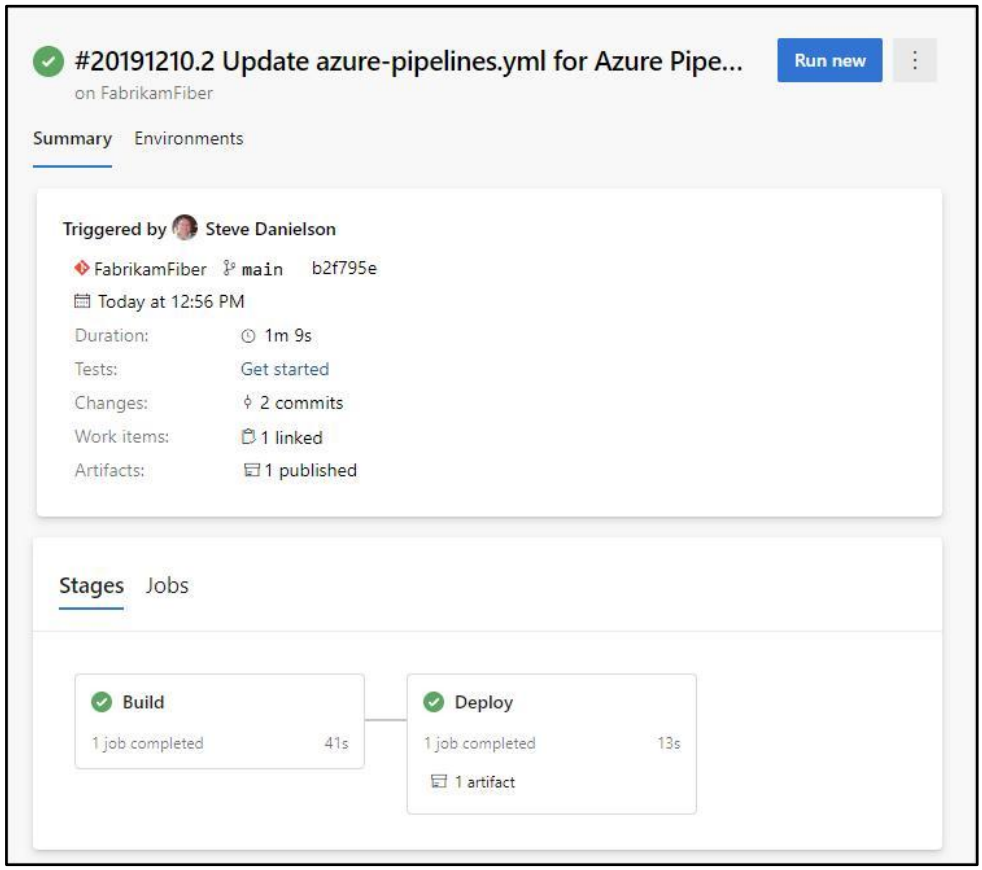
3. Screenshots



Screenshot 3.1: Select Source Repository



Screenshot 3.2: Pipeline YAML Editor



Screenshot 3.3: Pipeline Run Status

4. Expected Result / Outcome

A new Azure Pipeline is created and connected to the GitHub repository. Azure DevOps reads the azure-pipelines.yml file and executes the defined steps automatically

---

4. Building Sample Code Using Azure Pipeline

1. Objective

To build a sample application using Azure Pipelines and verify that the automated build process completes successfully.

2. Step-by-Step Procedure

1. Open the GitHub repository Azure-Pipeline-Demo.
2. Click Add file → Create new file.
3. Name the file app.js.
4. Enter the following sample JavaScript code:

   console.log("Hello, Azure DevOps Pipeline!");

   console.log("Sample application built successfully.");Commit the file to the repository.

5. Open Azure DevOps and select the pipeline created in Experiment 3.
6. Ensure that the pipeline contains the following YAML configuration, which runs the sample JavaScript file using Node.js:

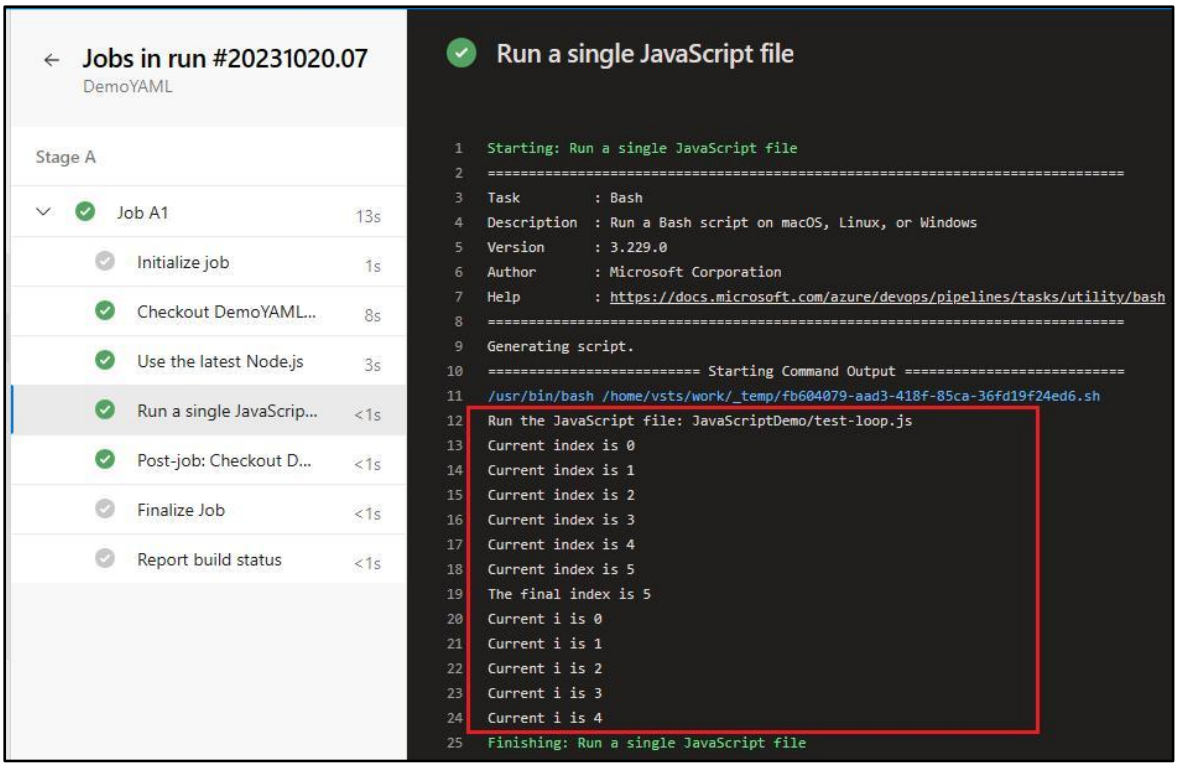
```
trigger:
    - main
pool:
    vmImage: 'ubuntu-latest'
steps:
  - task: NodeTool@0
    inputs:
      versionSpec: '20.x'
      displayName: 'Install Node.js'
      - script: node app.js
      displayName: 'Run sample application'
```

7. Save and commit the YAML file if you made changes.
8. Click Run pipeline if a run does not start automatically.
9. Select the appropriate branch, such as main, and start the run.
10. Open the running pipeline and select the job to view its logs.
11. Verify that the JavaScript output appears in the log and the pipeline finishes successfully.

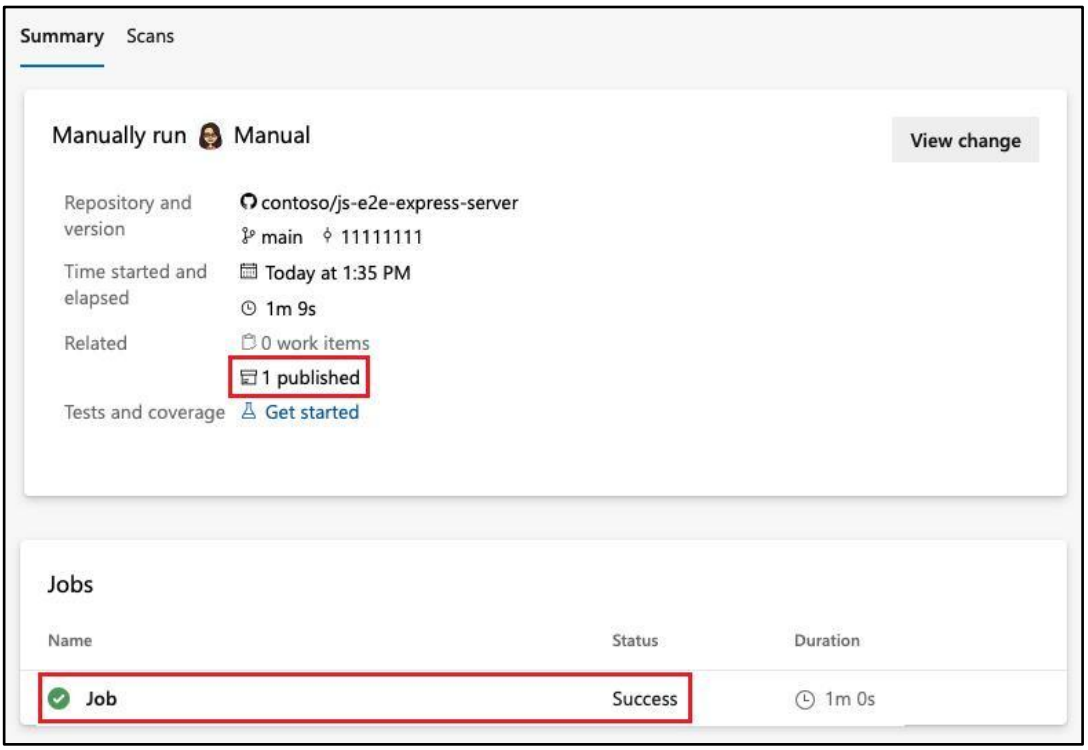
3. Screenshots



Screenshot 4.1: Sample Application Code



Screenshot 4.2: Build Execution Log



Screenshot 4.3: Successful Pipeline Run

4. Expected Result / Outcome

The sample JavaScript application executes successfully through Azure Pipelines. The pipeline log displays the program output, and the run completes with a Succeeded status.

5. Modifying the azure-pipelines.yml File

1. Objective

To modify the azure-pipelines.yml file to customize the pipeline configuration, define execution steps, and automate pipeline runs when changes are pushed to the repository.

2. Step-by-Step Procedure

1. Open the GitHub repository connected to Azure DevOps.
2. Locate the azure-pipelines.yml file in the repository root.
3. Click the file and select the edit option.
4. Modify the YAML file with the following configuration:

```
trigger:
        - main
pool:
        vmImage: 'ubuntu-latest'
variables:
        appName: 'AzurePipelineDemo'
steps:
    - task: NodeTool@0
    inputs:
        versionSpec: '20.x'
        displayName: 'Install Node.js'
- script: | echo "Building $(appName)"
        node app.js
    displayName: 'Build and run application'
- script: echo "Pipeline execution completed."
displayName: 'Completion message'
```

5. Save the changes and commit the updated file to the main branch.
6. Open Azure DevOps and navigate to Pipelines.
7. Select the pipeline connected to the repository.
8. Verify that the new configuration is used for the latest run.
9. If a run does not start automatically, click Run pipeline and select the main branch.
10. Open the job logs and verify the following:
    - Node.js is installed.

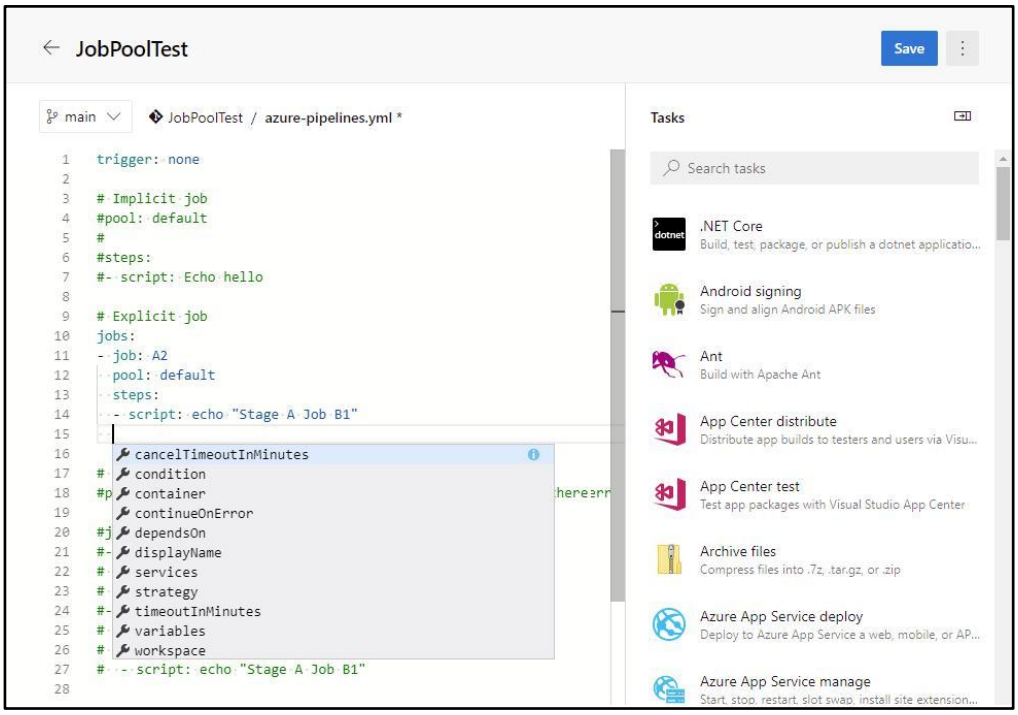
- The application name is printed.
- The JavaScript application executes.
- The completion message appears.

11. Confirm that the pipeline finishes successfully.

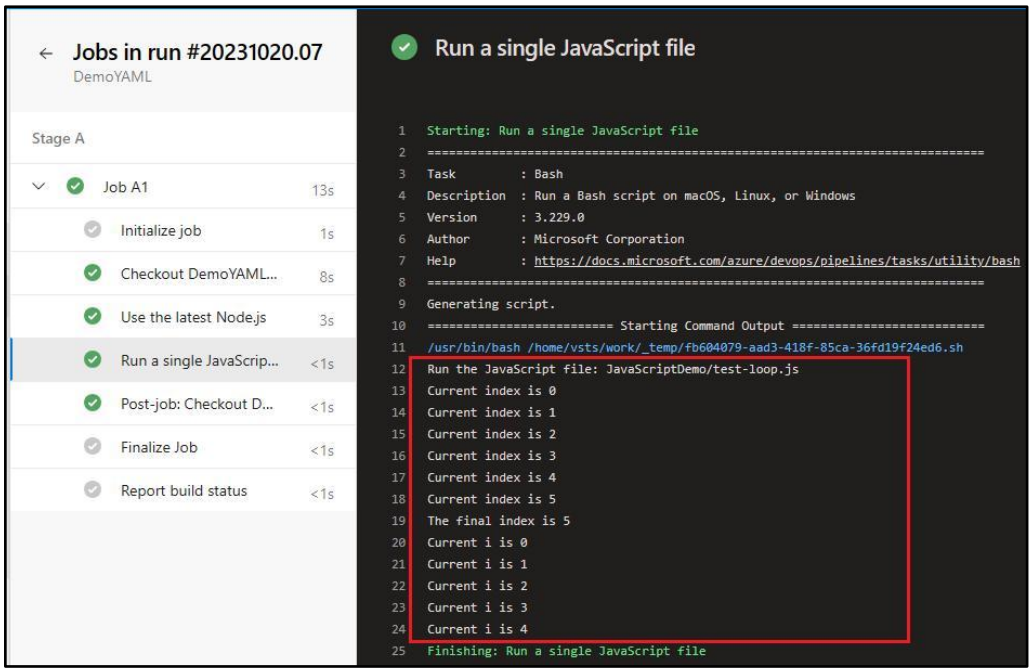
YAML element Purpose

trigger	Starts a pipeline run when changes are pushed to main.
pool	Selects an Ubuntu-hosted build agent.
variables	Defines a reusable application name.
NodeTool@0	Installs the specified Node.js version on the agent.
script	Executes shell commands on the agent.
displayName	Gives each pipeline step a readable name.

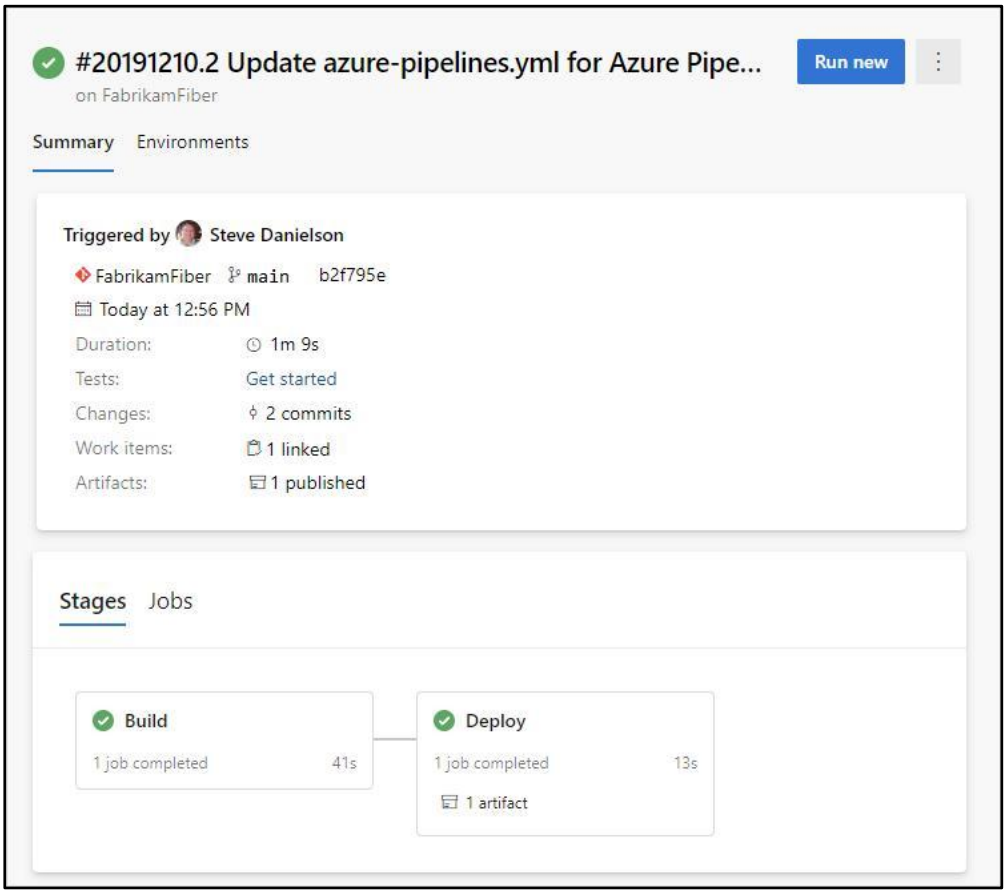
3. Screenshots



Screenshot 5.1: Modified YAML File



Screenshot 5.2: Execution Logs



Screenshot 5.3: Final Pipeline Status

4. Expected Result / Outcome

The azure-pipelines.yml file is modified successfully. The pipeline executes the updated steps, uses the defined variable, runs the sample application, and displays a completion message. Subsequent commits to the main branch trigger automated pipeline runs.